



MNIST Classification - Architecture and Normalization

Nikolas Nosál¹, Ondřej Studničný¹, and Jakub Vaněk¹

¹Mendel University in Brno, Faculty of Business and Economics

Abstract

Five neural network architectures — from a single linear layer to a residual network — are trained across five input normalization strategies on MNIST, yielding 25 controlled runs under identical hyperparameters. Models are built in PyTorch using Adam with cosine annealing and evaluated on accuracy, F1, MCC, and inference speed.

Keywords MNIST, neural networks, CNN, ResNet, normalization, classification, PyTorch

1. INTRODUCTION

Neural network performance depends on two largely independent decisions: the choice of architecture and the choice of input preprocessing. In practice these are often tuned together, making it difficult to attribute accuracy gains to either factor in isolation. This project separates the two by training five architectures — ranging from a single-layer linear network to a residual network — under five input normalization strategies, producing a fully controlled 5×5 grid of 25 independent runs on the MNIST handwritten digit dataset. All runs share identical hyperparameters, so any difference in outcome can be attributed directly to architecture or normalization rather than to training configuration.

1.a. Research Questions

1. Does input normalization affect final accuracy and convergence, and if so, by how much relative to architecture choice?
2. Which architecture achieves the best accuracy-to-parameter ratio on MNIST?

1.b. Goals

1. Implement five neural network architectures spanning the linear-to-residual design space.
2. Train each architecture under five input normalization strategies, producing 25 controlled runs.
3. Evaluate every run on accuracy, macro and weighted F1, MCC, parameter count, training time, and inference speed.
4. Identify which architecture–normalization combination generalizes best and at what computational cost.
5. Deploy the best-performing model in an interactive demo that accepts hand-drawn digits and returns live predictions.

ENC-NS PROJECT

Published May 17, 2026

Acknowledgements

Mendel University in Brno, Faculty of Business and Economics. All authors contributed equally to the experimental design, implementation, and analysis.

Data Availability

Source code, trained model checkpoints, and per-run metrics are available at the project repository. All 25 model–normalization combinations are fully reproducible with a single created CLI tool.

1.c. Dataset

MNIST (*Modified National Institute of Standards and Technology*) (LeCun et al., 1998) is the standard benchmark for handwritten digit classification. The dataset version used in this project is sourced from Johansson (2020).

Property	Value
Source	Kaggle – hojjatk/mnist-dataset
Total samples	70 000 grayscale images
Image size	28 × 28 px, 1 channel
Classes	10 (digits 0–9)
Train split	48 000 (80 % of official train set)
Validation split	12 000 (20 % of official train set)
Test split	10 000 (official test set, never seen during training)

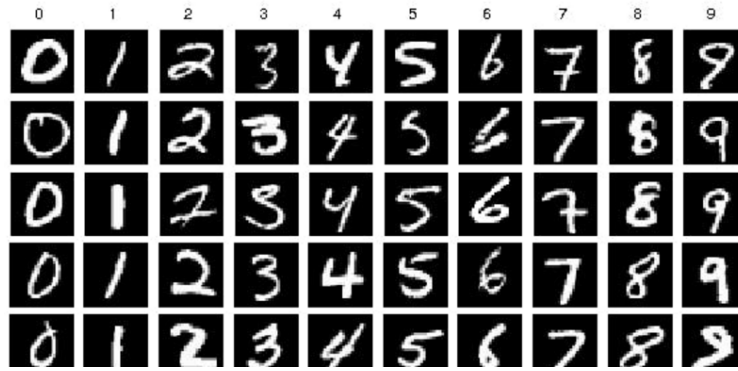


Figure 1: MNIST training set

1.d. Experiment at a Glance

The experiment forms a complete 5×5 grid – every architecture trained under every normalization strategy. No combination is skipped, ensuring a fair cross-comparison with no cherry-picking.

	LinearDNN	DeepDNN	SimpleCNN	DeepCNN	MiniResNet
none	●	●	●	●	●
minmax	●	●	●	●	●
standardize	●	●	●	●	●
symmetric	●	●	●	●	●
augmented	●	●	●	●	●

Each ● represents one independent training run – 25 in total. All runs use the same optimizer, learning rate, batch size, and number of epochs, so any difference in outcome is attributable solely to architecture or normalization.

2. ANALYSIS

This chapter covers the theoretical foundations of the experiment — the dataset, problem formulation, normalization strategies, neural network building blocks, and the evaluation metrics used to compare all 25 runs.

2.a. Dataset Selection

MNIST (Modified National Institute of Standards and Technology) (LeCun et al., 1998) is a benchmark dataset of 70 000 grayscale images of handwritten digits (0–9), each sized 28×28 pixels. Each image is a single-channel pixel grid with values in $[0, 255]$, where 0 is black and 255 is white. The label is an integer in $\{0, \dots, 9\}$ corresponding to the written digit. The dataset is pre-divided into 60 000 training images and 10 000 test images. For this project the training set is further split into 48 000 training and 12 000 validation samples using a fixed random seed, ensuring reproducibility across runs.

MNIST was selected as the experimental dataset for three reasons:

- **Controlled complexity** — images are small enough to train dozens of models on CPU without excessive compute, yet complex enough to differentiate architecture quality.
- **Established benchmark** — test accuracy on MNIST is well-documented across decades of literature, making results directly interpretable and comparable to published work.
- **Class balance** — all 10 digit classes are approximately equally represented ($\sim 7\,000$ samples each), which prevents class imbalance from confounding the comparison between architectures and normalization strategies.

2.b. Problem Classification

Digit recognition is a **supervised multiclass classification** problem. Each sample is a labeled pair $(\mathbf{x}_i, y_i)$, where $\mathbf{x}_i \in \mathbb{R}^{784}$ is the flattened pixel vector and $y_i \in \{0, \dots, 9\}$ is the ground-truth digit. The model learns a mapping $f : \mathbf{x} \mapsto y$ by minimizing categorical cross-entropy over the training set:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log P(y_i \text{ mid } \mathbf{x}_i) \quad (1)$$

The output layer produces 10 logits $z_0, \dots, z_9$, one per class. Softmax converts them to a probability distribution, and the predicted class is the highest-probability digit:

$$P(y = k \text{ mid } \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=0}^9 e^{z_j}}, \quad \hat{y} = \arg \max_k P(y = k \text{ mid } \mathbf{x}) \quad (2)$$

2.c. Input Normalization

Raw MNIST pixel values lie in $[0, 255]$ — a large magnitude range that causes disproportionately large gradients in the first layer and slows convergence. Five strategies are evaluated — one unnormalized baseline, three static transforms, and one augmentation-based variant — each implemented as a (train_transform, eval_transform) pair. For the first four both transforms are identical; for augmented, geometric perturbations apply only during training while validation and test always use standardization.

2.c.i. None — Unnormalized Baseline:

`ToTensor()` is applied and values are immediately scaled back to $[0, 255]$. This is the negative reference — it establishes what happens without any normalization and makes the benefit of the other strategies measurable.

$$x' = x \cdot 255 \quad (3)$$

2.c.ii. Min-Max:

`ToTensor()` alone, which divides pixel values by 255 and produces a $[0, 1]$ range:

$$x' = \frac{x}{255} \quad (4)$$

Preserves the original distribution shape. Removes the magnitude problem without shifting or centering the data.

2.c.iii. Standardization (Z-score):

Subtracts the MNIST dataset mean and divides by its standard deviation, producing zero-mean, unit-variance activations:

$$x' = \frac{x - \mu}{\sigma}, \quad \mu = 0.1307, \quad \sigma = 0.3081 \quad (5)$$

These statistics are computed over the full MNIST training set. Centering near zero keeps activations in the most sensitive region of ReLU and sigmoid nonlinearities, which is optimal for gradient flow.

2.c.iv. Symmetric:

Normalizes to $[-1, 1]$ using fixed statistics instead of the dataset-derived ones:

$$x' = \frac{x - 0.5}{0.5} = 2x - 1 \quad (6)$$

Equivalent to standardization with $\mu = 0.5, \sigma = 0.5$. Centered and symmetric, but slightly suboptimal because the fixed parameters do not match the true MNIST distribution.

2.c.v. Augmented:

Applies random geometric perturbations during training to artificially increase sample diversity, then standardizes. Validation and test use standardization only — no augmentation:

Transform	Parameters
RandomAffine — rotation	$\pm 10^\circ$
RandomAffine — translation	$\pm 10\%$ of image size
RandomAffine — scale	90–110%
Val/Test transform	Standardization ($\mu = 0.1307, \sigma = 0.3081$)

Augmentation forces the model to generalize across handwriting styles, orientations, and scales rather than memorizing exact pixel positions.

2.d. Neural Network Architectures

The five architectures in this project are built from four core building blocks — dense layers, convolutional layers, batch normalization, and residual connections. Understanding each component in isolation makes the architectural choices and their trade-offs easier to reason about.

2.d.i. Fully Connected Networks:

A fully connected (dense) layer maps every input feature to every output neuron:

$$\mathbf{y} = \sigma(W\mathbf{x} + \mathbf{b}) \quad (7)$$

where $W \in \mathbb{R}^{m \times n}$ is a learned weight matrix and σ is a nonlinear activation function. Applied to images, the input is first flattened into a vector, treating each pixel as an independent feature. This discards all spatial structure — the network has no way to know that neighboring pixels are related.

2.d.ii. Convolutional Networks:

A convolutional layer slides a small learned filter w across the spatial dimensions of the input, computing a dot product at each position:

$$(f * w)[i, j] = \sum_m \sum_n x[i + m, j + n] \cdot w[m, n] \quad (8)$$

Each filter detects a local pattern — an edge, a curve, a corner — and produces one feature map (LeCun et al., 1998). Two properties make convolutions far more parameter-efficient than dense layers for image data:

- **Weight sharing** — the same filter is applied at every spatial location, so the number of parameters depends on filter size, not image size.
- **Translation equivariance** — a pattern detected in one part of the image is recognized everywhere else.

2.d.iii. Batch Normalization:

Introduced by Ioffe & Szegedy (2015), batch normalization normalizes activations across a mini-batch to zero mean and unit variance, then re-scales them with learned parameters γ and β :

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \quad (9)$$

This prevents activations from drifting to extreme values during training, allowing higher learning rates and faster convergence. It also acts as a mild regularizer. In this project, BatchNorm is used in DeepDNN, DeepCNN, and MiniResNet — but deliberately omitted from LinearDNN and SimpleCNN, which serve as unnormalized baselines.

2.d.iv. Residual Connections:

Deep networks suffer from vanishing gradients — by the time the loss signal propagates back through many layers, it becomes too small to update early weights effectively. Residual connections, introduced in ResNet (He et al., 2016), solve this by adding the block's input directly to its output:

$$\text{output} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \quad (10)$$

The network learns the residual — the correction to the identity — rather than a full mapping from scratch. The skip connection gives gradients a direct path to earlier layers regardless of depth. This is used exclusively in MiniResNet.

2.d.v. Global Average Pooling:

After the convolutional blocks, the feature map has shape (C, H, W) — C channels, each of size $H \times W$. Global Average Pooling collapses each channel to a single scalar by averaging across its spatial extent:

$$\text{GAP}(f)_k = \frac{1}{H \times W} \sum_{i,j} f_k[i, j] \quad (11)$$

This produces a vector of length C that summarizes what the network detected, without caring where it detected it. All three CNN architectures (SimpleCNN, DeepCNN, MiniResNet) use GAP before the final classification layer, which significantly reduces parameter count compared to flattening the full feature map.

2.e. Evaluation Metrics

All classification metrics are derived from the **confusion matrix** C , where C_{ij} is the number of samples with true class i that were predicted as class j . The diagonal entries C_{ii} are correct predictions; off-diagonal entries are mistakes. A perfect classifier has a purely diagonal confusion matrix.

2.e.i. Accuracy:

The overall fraction of correct predictions across all 10 classes:

$$\text{Acc} = \frac{\sum_i C_{ii}}{\sum_{i,j} C_{ij}} \quad (12)$$

Reliable when classes are balanced, which holds for MNIST. Per-class accuracy $\text{Acc}_i = C_{ii} / \sum_j C_{ij}$ is also computed for each digit independently, revealing which specific digits the model struggles with most regardless of overall performance.

2.e.ii. Precision, Recall, F1:

Accuracy alone hides how errors are distributed. Precision and recall break this down per class:

- **Precision** — of all samples the model predicted as class i , how many actually were class i ? High precision means few false alarms.
- **Recall** — of all samples that truly are class i , how many did the model catch? High recall means few misses.

$$\text{Precision}_i = \frac{C_{ii}}{\sum_j C_{ji}}, \quad \text{Recall}_i = \frac{C_{ii}}{\sum_j C_{ij}} \quad (13)$$

A model can trivially maximize one at the expense of the other — predicting everything as class i gives perfect recall but terrible precision. F1 is their harmonic mean and penalizes this imbalance:

$$F1_i = \frac{2 \cdot \text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \quad (14)$$

Both **macro** (unweighted mean across classes) and **weighted** (mean weighted by class support) averages are reported. For balanced datasets like MNIST the two are nearly identical, but macro is more sensitive to rare-class failures.

2.e.iii. *Matthews Correlation Coefficient (MCC)*:

Accuracy and F1 can still look good on a model that systematically confuses two specific classes. MCC avoids this by treating the confusion matrix as a correlation problem — it measures how well the predicted label distribution matches the true one across all classes at once (Gorodkin, 2004):

$$\text{MCC} = \frac{N \sum_k C_{kk} - \sum_k p_k s_k}{\sqrt{(N^2 - \sum_k p_k^2)(N^2 - \sum_k s_k^2)}} \quad (15)$$

where N is total samples, $p_k = \sum_j C_{kj}$ is the row sum (all true instances of class k), and $s_k = \sum_j C_{jk}$ is the column sum (all predictions of class k). The scale is intuitive: $\text{MCC} = 1$ is a perfect classifier, $\text{MCC} = 0$ is no better than random guessing, $\text{MCC} = -1$ is perfectly wrong on every sample. It is the primary scalar used to rank all 25 runs in the results.

3. IMPLEMENTATION

The experiment is implemented in Python 3.12 using PyTorch 2.8 as the core deep learning framework. All model definitions, training logic, data pipelines, and evaluation are contained in four source files under `src/`. Training, evaluation, and inference code is written directly against the PyTorch API. The interactive demo is a separate Jupyter notebook in `example/`.

3.-i. Dependencies:

Dependencies are managed with Poetry and pinned in `requirements.txt` for pip compatibility:

Package	Role
<code>torch, torchvision</code>	Model training, transforms, DataLoaders, MNIST download
<code>numpy, matplotlib</code>	Metrics computation and report plots
<code>typer, rich</code>	CLI interface and formatted terminal output
<code>ipycanvas, ipywidgets</code>	Interactive drawing widget in the demo notebook
<code>tqdm</code>	Per-epoch and per-batch progress bars
<code>Pillow</code>	Image loading for the predict CLI command

3.-ii. Installation:

Clone the repository and install dependencies using either Poetry or pip:

```
# Poetry (recommended)
$ poetry install

# pip
$ pip install -r requirements.txt
```

3.-iii. Project Structure:

The codebase is intentionally flat — four source files in `src/` cover the entire experiment pipeline. All commands in this document assume the working directory is the project root.

```
src/
├─ data.py      – MNIST download, normalization strategies, DataLoader factory
├─ models.py   – ModelFactory registry + all 5 architecture definitions
├─ train.py    – Trainer (fit/evaluate/save) + Detector (inference)
├─ main.py     – Typer CLI: train / report / predict
example/
├─ canvas.py   – Jupyter drawing widget (ipycanvas + PyTorch inference)
└─ demo.ipynb  – Interactive digit prediction notebook
models/
  – Saved checkpoints (.pt) and metrics (.json) per run
output/
  – Generated plots and reports
docs/
  – MyST documentation source – the document you are reading
```

3.a. Data Preparation

All data handling lives in `src/data.py`. The `MNISTData` class downloads the dataset on first run, applies the selected normalization strategy, splits the training set into training and validation subsets, and returns three ready-to-use `DataLoaders`. The same class is used for all 25 runs — only the normalization config changes between them, keeping everything else identical.

3.a.i. Splitting:

The official MNIST training set (60 000 samples) is split into training and validation subsets. `MNISTData` computes the split size dynamically from a configurable parameter — defaulting to 20%.

```
n_val = int(val_split * len(full_train))    # 0.20 × 60 000 = 12 000
n_train = len(full_train) - n_val           # 48 000
random_split(full_train, [n_train, n_val], generator=torch.Generator().manual_seed(42))
```

The fixed seed makes the split identical across all 25 runs — every model and normalization combination trains and validates on exactly the same samples. The official test set (10 000 samples) is kept entirely separate — it is never seen during training or validation and is only used once per run for final evaluation.

3.a.ii. Normalization:

Each normalization strategy is defined as a `(train_transform, eval_transform)` pair. For all static strategies the two are identical. For augmented they differ — geometric perturbations apply only during training; validation and test always use standardization to ensure fair evaluation:

```
"none":      ToTensor() → ×255           # raw pixels, negative reference
"minmax":    ToTensor()                 # [0, 1]
"standardize": ToTensor() → Normalize(0.1307, 0.3081)
"symmetric": ToTensor() → Normalize(0.5, 0.5)
"augmented": RandomAffine(10°, ±10%, 90–110%) → Normalize(0.1307, 0.3081)
              eval: same as standardize
```

The validation subset is explicitly constructed with the eval transform regardless of what the training transform is — this is handled in `MNISTData._loaders()`.

3.a.iii. DataLoaders:

Three `DataLoaders` are constructed per run — one for each split. The training loader shuffles samples each epoch; validation and test do not:

```
Dataloader(train_ds, batch_size=512, shuffle=True)
Dataloader(val_ds,   batch_size=256, shuffle=False)
Dataloader(test_ds,  batch_size=256, shuffle=False)
```

The training batch is larger (512) to reduce the number of gradient updates per epoch and speed up training. Validation and test use 256 — no gradient storage is needed so a smaller batch is sufficient.

3.b. Model Architectures

All five models are defined in `src/models.py` registered in `ModelFactory` — a simple decorator-based registry that maps a name string to a constructor. Calling `ModelFactory.create("DeepCNN")` returns a fresh instance without the training code needing to import it directly. Every model outputs 10 logits, one per digit class.

3.b.i. LinearDNN — Baseline:

The simplest possible architecture: one hidden layer, no convolutions, no regularization. Every pixel is treated as an independent feature — spatial relationships between neighboring pixels are completely ignored. Its role is to establish a lower bound on what a non-spatial model can achieve.

`Flatten(784) → Linear(784→128) → ReLU → Linear(128→10)`

3.b.ii. DeepDNN — Deep Dense Network:

Adds depth and regularization to the dense baseline. Three hidden layers with `BatchNorm` after each projection and `Dropout` between layers — this tests whether increased capacity and regularization can close the gap to convolutional models while still discarding spatial structure.

`Flatten(784)`
 → `Linear(784→512)` → `BatchNorm1d` → `ReLU` → `Dropout(0.3)`
 → `Linear(512→256)` → `BatchNorm1d` → `ReLU` → `Dropout(0.3)`
 → `Linear(256→128)` → `BatchNorm1d` → `ReLU`
 → `Linear(128→10)`

3.b.iii. SimpleCNN — Convolutional Baseline:

Minimal convolutional model — two conv blocks with `noBatchNormorDropout`. Introduces spatial awareness through learned filters while keeping the architecture as bare as possible, establishing the lower bound for CNN performance in this comparison.

`Conv2d(1→32, 3×3, pad=1) → ReLU → MaxPool2d(2)` # $28 \times 28 \rightarrow 14 \times 14$
`Conv2d(32→64, 3×3, pad=1) → ReLU → MaxPool2d(2)` # $14 \times 14 \rightarrow 7 \times 7$
`AdaptiveAvgPool2d(1) → Flatten(64)`
 → `Linear(64→128) → ReLU → Linear(128→10)`

3.b.iv. DeepCNN — Deep Convolutional Network:

Extends SimpleCNN with a third convolutional block, `BatchNorm` after each convolution, and `Dropout` in the classifier head. VGG-inspired — deeper feature extraction with regularization at every stage.

`Conv2d(1→32, pad=1) → BatchNorm2d → ReLU → MaxPool2d(2)` # $28 \times 28 \rightarrow 14 \times 14$
`Conv2d(32→64, pad=1) → BatchNorm2d → ReLU → MaxPool2d(2)` # $14 \times 14 \rightarrow 7 \times 7$
`Conv2d(64→128, pad=1) → BatchNorm2d → ReLU` # 7×7 , no pool
`AdaptiveAvgPool2d(1) → Flatten(128)`
 → `Linear(128→256) → ReLU → Dropout(0.5)`
 → `Linear(256→128) → ReLU`
 → `Linear(128→10)`

3.b.v. *MiniResNet — Residual Network:*

The most complex architecture. Residual (skip) connections let the network learn corrections to the identity rather than a full mapping, giving gradients a direct path through the network and preventing them from vanishing in deeper layers.

```

Stem:  Conv2d(1→32, pad=1) → BatchNorm → ReLU          # 28×28

Layer1: ResBlock(32) → MaxPool2d(2)                    # 28×28 → 14×14
Layer2: Conv2d(32→64, pad=1) → BatchNorm → ReLU
        ResBlock(64) → MaxPool2d(2)                    # 14×14 → 7×7

Head:  AdaptiveAvgPool2d(1) → Flatten(64) → Linear(64→10)

```

Each ResBlock is two conv layers with a skip connection: Conv→BN→ReLU→Conv→BN, output = $F(x) + x$ → ReLU. The head uses Global Average Pooling to collapse the spatial dimensions to one value per channel before classification — this replaces a large dense layer and keeps parameter count low compared to DeepCNN despite the additional residual blocks.

3.c. *Training Pipeline*

All training logic lives in `src/train.py`. The Trainer class wraps a model, device, and hyperparameters, exposes a `fit()` method that runs the full training loop, and returns a History object with per-epoch loss and accuracy for both training and validation sets. Hyperparameters are fixed across all 25 runs — EPOCHS = 15, LR = 1e-3, BATCH = 512.

3.c.i. *Optimizer and Scheduler:*

```

optimizer = Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=epochs)
criterion = CrossEntropyLoss()

```

CosineAnnealingLR decays the learning rate smoothly from `lr` to 0 over the full 15-epoch run:

$$\eta_t = \frac{\eta_{\max}}{2} \left(1 + \cos\left(\frac{t\pi}{T_{\max}}\right) \right) \quad (16)$$

This avoids abrupt step-decay drops — the model converges gradually into the loss minimum rather than oscillating around it. `weight_decay=1e-4` adds L2 regularization directly through Adam.

3.c.ii. *Mixed Precision and Compilation:*

On CUDA, the model is compiled at initialization with `torch.compile`, which fuses operations into optimized kernels via TorchInductor. During each forward pass, bfloat16 mixed precision is enabled:

```

with torch.autocast(device.type, dtype=torch.bfloat16, enabled=device.type == "cuda"):
    logits = model(xb)
    loss = criterion(logits, yb)

```

bfloat16 halves memory bandwidth while preserving the dynamic range of float32. On CPU autocast is disabled and the model runs in eager mode at full precision.

3.c.iii. Metrics Computed After Each Run:

Metric	Description
test_acc / test_loss	Final evaluation on the held-out test set
per_class_acc	Per-digit accuracy from the confusion matrix diagonal
precision / recall / f1	Per-class and macro/weighted averages
mcc	Multiclass Matthews Correlation Coefficient
params	Total trainable parameter count
train_time_s	Wall-clock training duration in seconds
inference_ms_per_sample	Average forward-pass time per sample on the test set
ckpt_size_kb	Checkpoint file size in kilobytes

3.d. CLI

The experiment workflow is exposed through a CLI in `src/main.py`. Run from the project root:

```
python src/main.py <command> # module (installed with pip/poetry)
mnist-exp <command>         # installed entry point (only with poetry)
```

3.d.i. train:

Trains one or more model $\times$ normalization combinations and saves results to `models/`:

```
python src/main.py train --model SimpleCNN --norm augmented # single run
python src/main.py train -m DeepCNN -m MiniResNet -n standardize # 2x1 grid
python src/main.py train                                     # all 25
```

Hyperparameters can be overridden: `--epochs`, `--lr`, `--batch`. Each run saves `{Model}_{norm}.pt` (weights) and `{Model}_{norm}.json` (metrics) to `models/`.

3.d.ii. report:

Reads saved JSON files and generates reports — no re-training required:

```
python src/main.py report all # ranked tables + heatmaps
python src/main.py report SimpleCNN_augmented # single-model breakdown
```

3.d.iii. Predict command:

Runs inference on a single image and prints per-digit probabilities:

```
python src/main.py predict digit.png # best saved model
python src/main.py predict digit.png --model MiniResNet_augmented
```

3.d.iv. Default (no command):

Running `python src/main.py` without a subcommand trains all 25 combinations in sequence and then generates a full comparison report — equivalent to `train` followed by `report all`.

3.d.v. Interactive Demo:

The `example/demo.ipynb` notebook provides an interactive drawing widget implemented in `example/canvas.py`. Draw a digit in the browser and see live predictions from the best saved model.

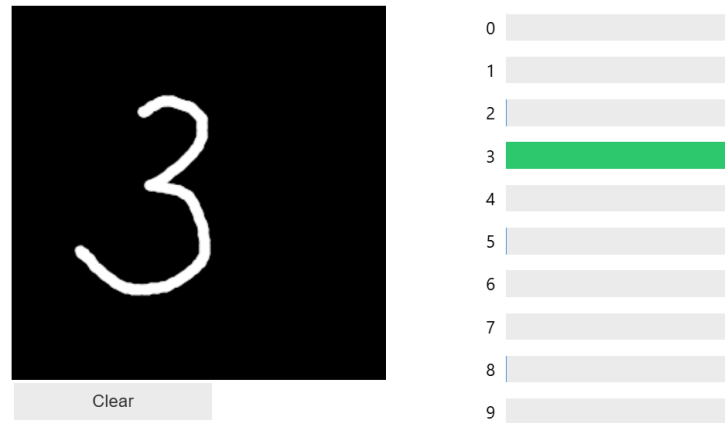


Figure 2: Interactive digit recognition demo — digit 3 drawn on the canvas.

4. EVALUATION & DISCUSSION

All metrics are computed on the official MNIST test set (10 000 samples, never seen during training or validation). Each of the 25 runs uses identical hyperparameters — only the architecture and normalization strategy vary. Results are reported as test accuracy, weighted F1, and Matthews Correlation Coefficient (MCC).

4.a. Full Grid Ranking

The results split cleanly by architecture, not by normalization. MiniResNet leads all 25 runs (99.38–99.58%), with the best single result being MiniResNet trained on augmented data at 99.58% test accuracy. DeepCNN and DeepDNN cluster tightly in the high-98% range, and LinearDNN sits a full percentage point lower — a gap that no normalization strategy can close. The choice of how to scale the inputs barely moves the needle for any of these four; the ceiling is set by the architecture itself.

SimpleCNN is the exception. Without BatchNorm, its performance swings nearly 20 points depending solely on input scale — from 96.15% with raw pixels down to 76.87% with min-max normalization. Every other architecture stays within 2 points across all five strategies. That contrast is the clearest finding in the entire grid: normalization matters only when the architecture gives it room to matter.

Accuracy, weighted F1, and MCC rank all 25 runs in essentially the same order, confirming that on a balanced ten-class dataset, any of the three is a reliable summary of model quality.

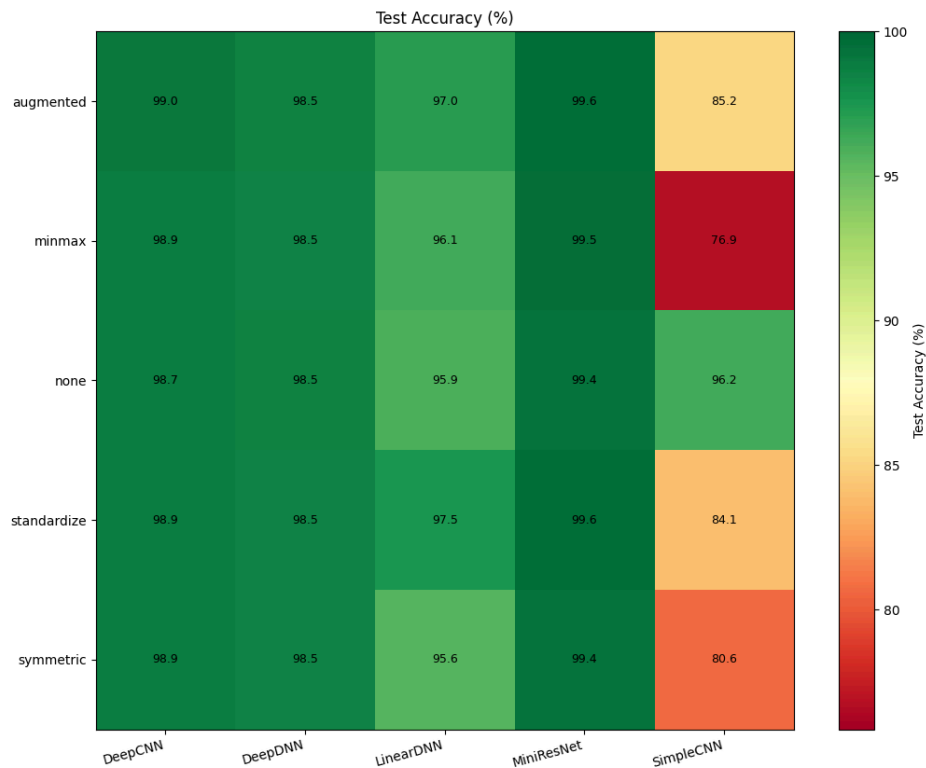


Figure 3: Test accuracy (%) for all 25 model–normalization combinations.

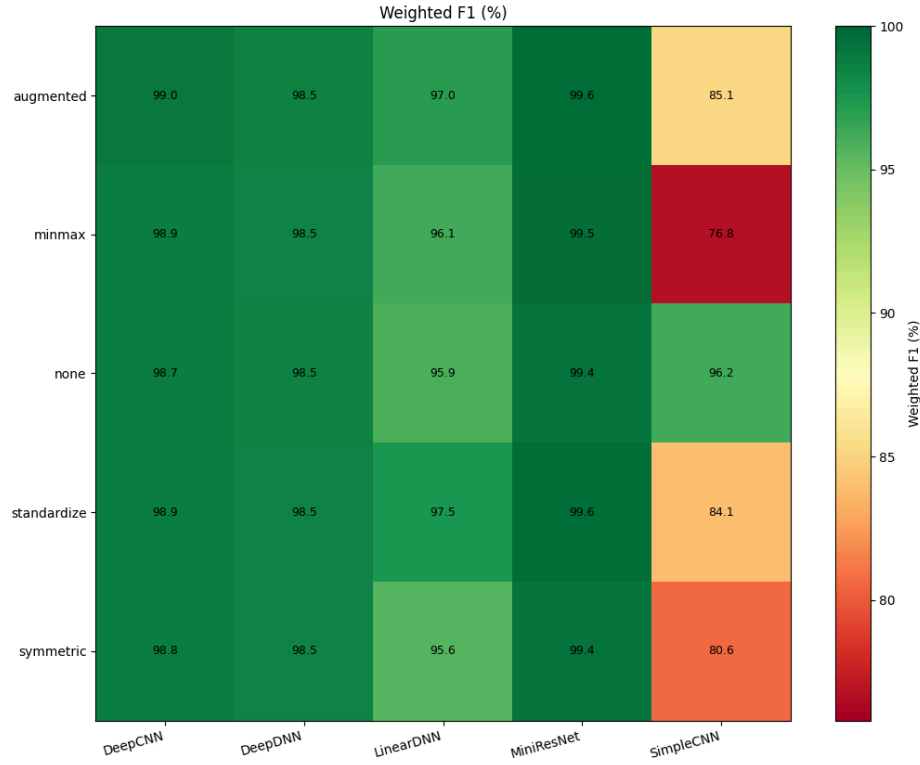
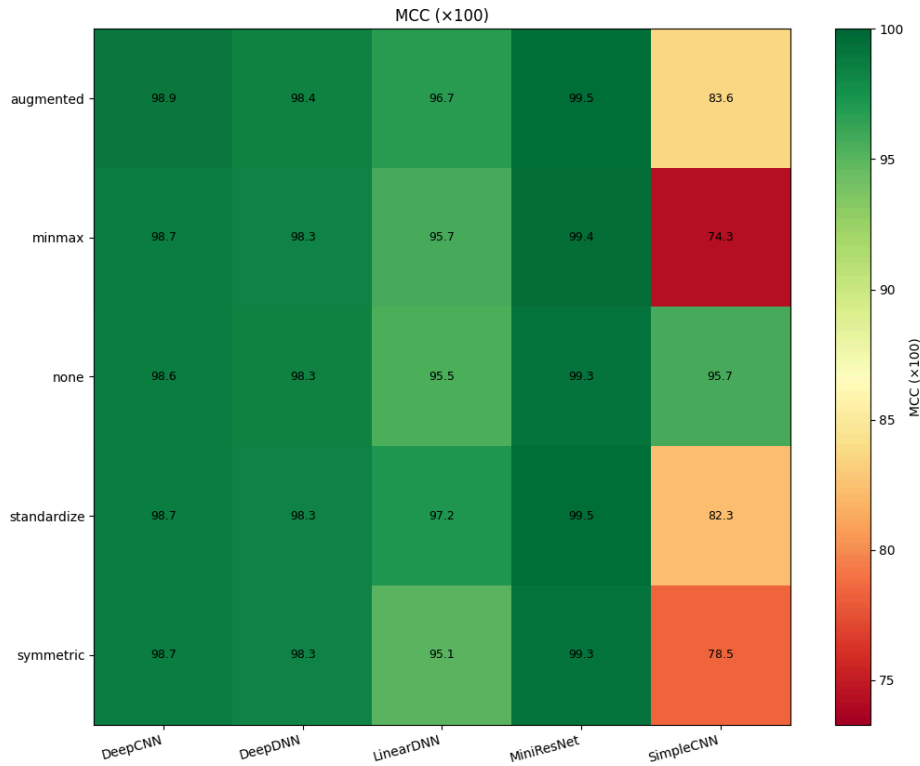


Figure 4: Weighted F1 (%) for all 25 model-normalization combinations.

Figure 5: MCC ($\times 100$) for all 25 model-normalization combinations.

4.b. Model Comparison

Model	Params	Avg Acc	Best	Best Norm	Worst	Worst Norm	Spread
MiniResNet	112,394	99.48%	99.58%	augmented	99.38%	symmetric	0.20%
DeepCNN	160,330	98.86%	99.02%	augmented	98.71%	none	0.31%
DeepDNN	569,226	98.49%	98.54%	augmented	98.47%	minmax	0.07%
LinearDNN	101,770	96.43%	97.46%	standardize	95.57%	symmetric	1.89%
SimpleCNN	28,426	84.59%	96.15%	none	76.87%	minmax	19.28%

Four of the five architectures include BatchNorm, and for those four the **Spread** column is almost irrelevant — swapping normalization strategies moves accuracy by at most 0.31 points. The normalization layer inside the network corrects for input scale before activations reach the deeper layers, so it barely matters how the pixels arrive. LinearDNN is the outlier among them at 1.89 points, but even that is small compared to what actually separates the models from each other.

4.b.i. SimpleCNN anomaly:

SimpleCNN is in a different category entirely. Its 19-point swing between none (96.15%) and minmax (76.87%) is not a normalization problem — it is a BatchNorm problem. When min-max scaling compresses pixel values to $[0, 1]$, the filters in the first convolutional layer receive signals too small to respond to meaningfully. Raw $[0, 255]$ values restore enough activation magnitude for the filters to fire. Without BatchNorm to compensate, the model is fully exposed to whatever the input pipeline hands it — and the confusion matrix below shows what that costs: digits 2, 4, 5, and 9 all fall below 75% F1, with classes that should be separable bleeding into one another because the features that distinguish them never get activated.

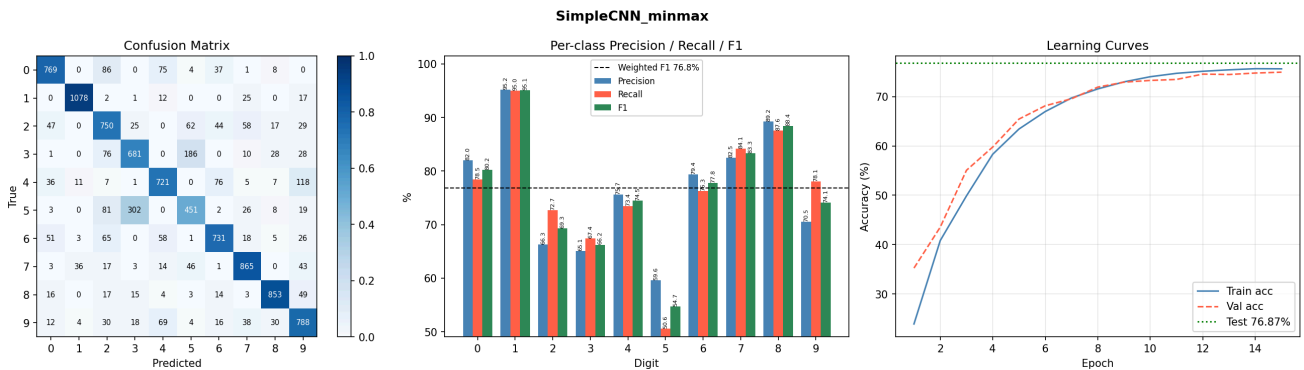


Figure 6: SimpleCNN (minmax) — the worst run (76.87%).

4.c. Normalization Comparison

Norm	Avg Acc (4 models)	Best single result
standardize	98.59%	99.56% (MiniResNet)
augmented	98.55%	99.58% (MiniResNet)
minmax	98.24%	99.48% (MiniResNet)
none	98.13%	99.39% (MiniResNet)
symmetric	98.07%	99.38% (MiniResNet)

SimpleCNN is excluded here — its BatchNorm-driven anomaly would dominate the averages and obscure any signal from normalization itself.

Among the four BatchNorm architectures, the differences between strategies are small. Standardization edges out the rest on average (98.59%) because using the exact dataset statistics ($\mu = 0.1307$, $\sigma = 0.3081$) centres activations in the optimal range for gradient flow. Augmentation produces the single best result (99.58%, MiniResNet) by forcing the model to generalise across rotations, translations, and scale variations — but it pulls down the average because it actively hurts LinearDNN. LinearDNN_standardize (97.46%) outperforms LinearDNN_augmented (97.05%): geometric perturbations are only useful to a model that understands spatial structure, and one that flattens all pixels into a vector cannot exploit them. The gap between the top and bottom strategy is just 0.52 points — on architectures with BatchNorm, normalization is a secondary concern.

4.d. Efficiency

Model	Params	Avg train/run (s)	Avg inference (ms/sample)	Checkpoint (KB)	Best Acc
SimpleCNN	28,426	951	0.239	115	96.15%
LinearDNN	101,770	838	0.009	400	97.46%
MiniResNet	112,394	2,259	0.672	456	99.58%
DeepCNN	160,330	994	0.339	637	99.02%
DeepDNN	569,226	718	0.034	2,238	98.54%

The table compares models on three dimensions averaged across all five normalization runs: parameter count, wall-clock training time, and inference speed. Parameter count and checkpoint size are fixed per architecture regardless of normalization.

MiniResNet is the most parameter-efficient model — with 112k parameters it outperforms DeepCNN (160k, 99.02%) and DeepDNN (569k, 98.54%), the latter using $5\times$ the parameters for 1% lower accuracy. Residual connections allow deep gradient flow without stacking extra layers, and Global Average Pooling replaces a large dense classifier with a single value per channel. The scatter plot below puts this in perspective: MiniResNet sits at the Pareto frontier of the accuracy–efficiency trade-off, achieving the highest accuracy at a lower parameter cost than both heavier models.

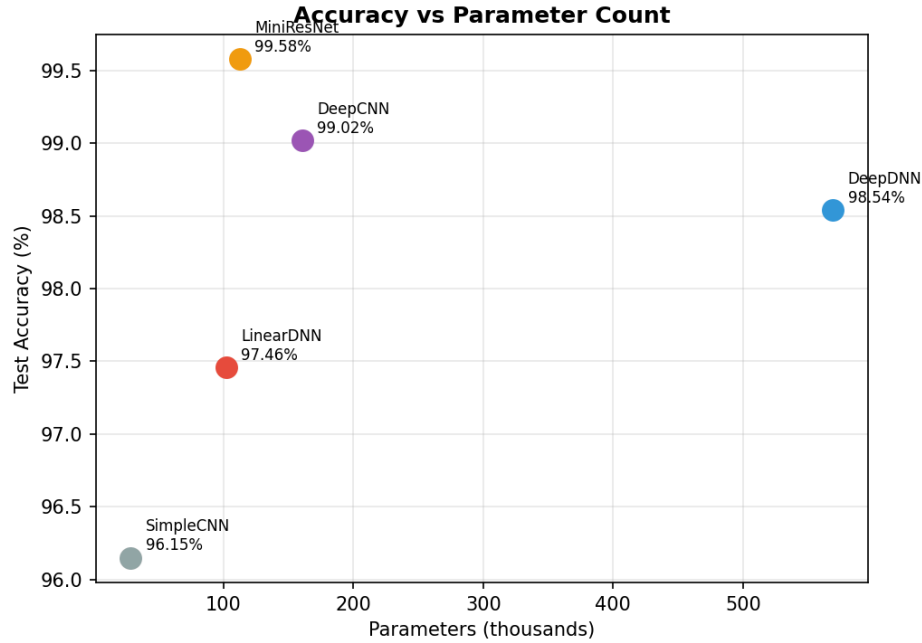


Figure 7: Test accuracy vs parameter count for each architecture’s best run.

The one area where MiniResNet pays a real cost is training time ($\sim 2\,259$ s/run on CPU), driven by its two residual blocks. LinearDNN sits at the opposite extreme — $75\times$ faster at inference — but is capped at 97.46% by its inability to exploit spatial structure. For latency-critical applications where that accuracy ceiling is acceptable, LinearDNN is the practical choice.

The learning curves tell the same story. DeepDNN and MiniResNet reach near-ceiling accuracy within the first five epochs and show no overfitting, while LinearDNN converges more slowly and plateaus below the CNN models. SimpleCNN_none converges but at a much lower ceiling — the limit is not training duration but the absence of regularization. Within the BatchNorm architectures, normalization has no meaningful effect on convergence speed: all five strategies reach their respective ceilings within the same epoch range for a given architecture.

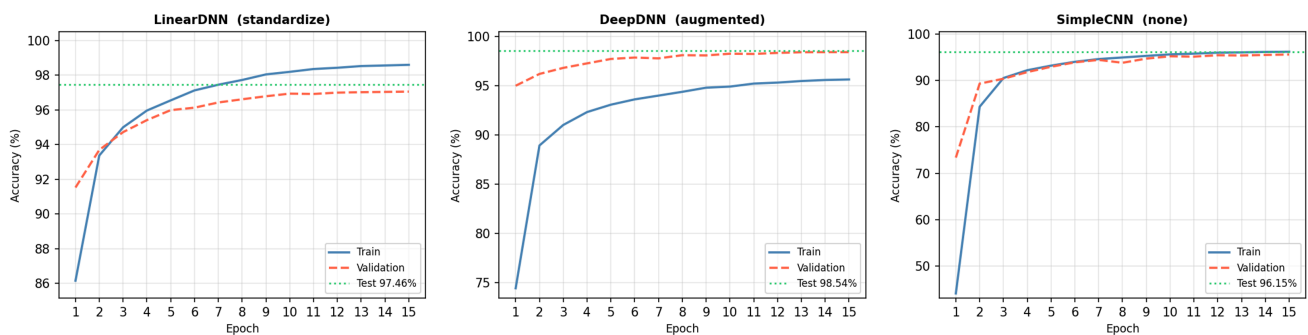


Figure 8: Learning curves for LinearDNN (standardize), DeepDNN (augmented), and SimpleCNN (none).

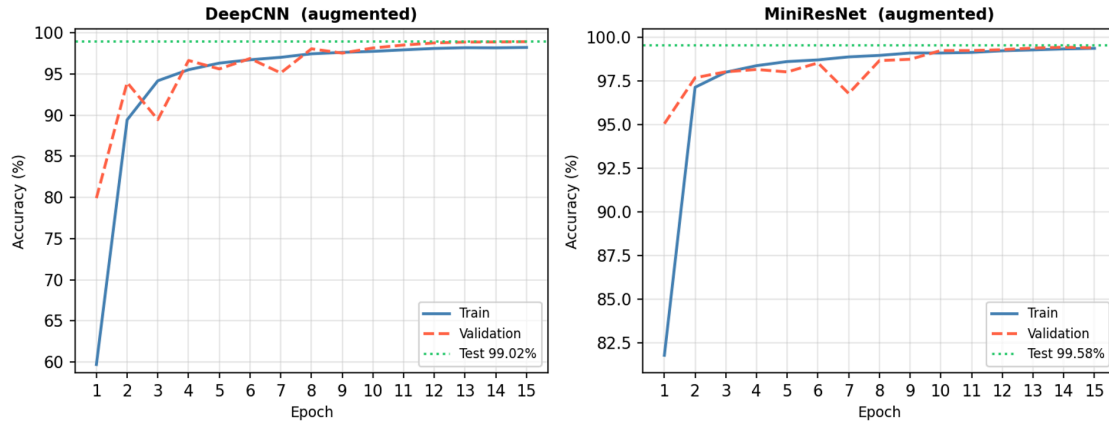


Figure 9: Learning curves for DeepCNN (augmented) and MiniResNet (augmented).

4.e. Per-digit Analysis

Two patterns hold across every architecture. Digit 1 is consistently the easiest — its narrow vertical stroke is distinctive enough that even a model with no spatial awareness can separate it reliably. Digit 5 is consistently the hardest, most often confused with 6 and 9 due to shared curved strokes; it has the lowest recall in the best-performing model and the widest error spread in the weakest one.

SimpleCNN breaks the pattern in a revealing way. While its digit 1 F1 stays near 99%, digits 2, 5, and 9 fall below 95% — a spread far larger than any other architecture shows. Without BatchNorm, activations vary wildly depending on stroke thickness and pixel density, so the model happens to learn some digits well and fails systematically on others. It is not a difficulty problem — it is a normalization problem expressed at the per-class level.

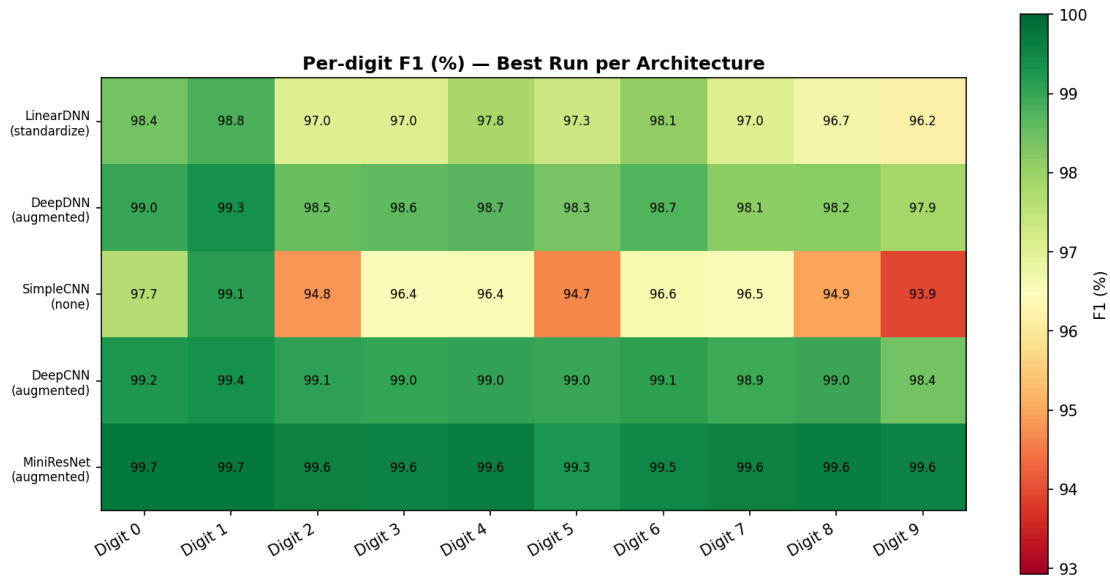


Figure 10: Per-digit F1 (%) for each architecture's best run.

4.f. Best Model – MiniResNet augmented

MiniResNet trained on augmented data is the strongest configuration across all 25 runs: 99.58% test accuracy, MCC = 0.9953, macro and weighted F1 both at 0.9957–0.9958 — the negligible difference between the two is expected on a balanced dataset. More telling than the aggregate numbers is the per-class breakdown: every single digit exceeds 99% F1, and the confusion matrix is nearly diagonal.

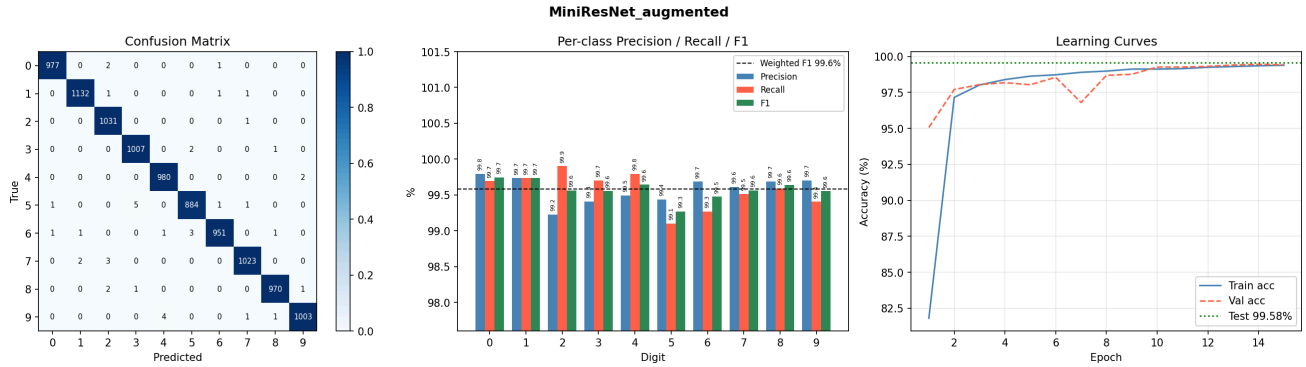


Figure 11: MiniResNet (augmented) – Confusion matrix, per-class precision/recall/F1, and learning curves.

5. CONCLUSION

This project trained five neural network architectures under five input normalization strategies — 25 runs in total — to isolate how much each factor independently affects MNIST digit classification accuracy. The controlled setup makes the answer unusually clear: architecture is the dominant factor, and normalization is secondary.

The spread between the best and worst architecture (MiniResNet at 99.58% vs. LinearDNN at 97.46%) is 2.12 points. The largest normalization-induced swing within a single architecture is 1.89 points — and that is LinearDNN, an outlier. For every CNN architecture, swapping normalization strategies moves accuracy by under 0.35 points. The ceiling is set by the architecture, not by how pixels are scaled.

The one exception that proves the rule is SimpleCNN. Its 19-point swing between none and minmax is not a normalization effect — it is what happens when BatchNorm is absent. Without it, the network has no internal mechanism to correct for input scale, so the preprocessing pipeline becomes the dominant factor. Every architecture with BatchNorm is effectively immune to this; SimpleCNN is fully exposed to it.

On the efficiency side, MiniResNet achieves the highest accuracy with just 112k parameters — outperforming DeepDNN at one-fifth the size. Residual connections give gradients a direct path to early layers, removing the need for brute-force depth. Among normalization strategies, standardization has the best average while augmentation produces the single best result; for spatially-aware models, augmentation is the stronger choice.

The recommended configuration is MiniResNet with augmented normalization: 99.58% test accuracy, MCC = 0.9953, no digit below 99% F1, and compact enough to run at interactive speeds as demonstrated in the digit recognition demo.

6. REFERENCES

- Gorodkin, J. (2004). Comparing two K-category assignments by a K-category correlation coefficient. *Computational Biology and Chemistry*, 28(5–6), 367–374.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 448–456.
- Johansson, H. (2020). *MNIST Dataset* [Data set]. Kaggle.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.